

“1. Contest Problems Upsolving”

A. Beautiful Pairs

Problem Statement:

You are given two arrays A and B, each containing n integers. A pair is considered beautiful if an element from A matches an element from B and each index can be used only once (pairwise disjoint). You are allowed to change exactly one element in array B to maximize the number of such beautiful pairs. The task is to determine the maximum number of pairwise disjoint beautiful pairs possible after this change

Theory/Approach:

The approach is to first count how many elements can already form matching pairs. Both arrays are sorted so that matching elements can be efficiently found using a two-pointer technique. We traverse both arrays and count the number of equal elements (matches). If all elements are already matched, changing one element will reduce the count by one, so the answer becomes matches - 1. Otherwise, we can increase the number of matches by changing one unmatched element in B, so the answer becomes matches+1

Code:

```
G++ a.cpp > ...
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      int n;
7      cin >> n;
8
9      int A[n], B[n];
10
11     for (int i = 0; i < n; i++)
12     {
13         cin >> A[i];
14     }
15     for (int i = 0; i < n; i++)
16     {
17         cin >> B[i];
18     }
19
20     sort(A, A + n);
21     sort(B, B + n);
22
23     int i = 0, j = 0;
24     int matches = 0;
25
26     while (i < n && j < n)
27     {
28         if (A[i] == B[j])
29         {
30             matches++;
31             i++;
32             j++;
33         }
34         else if (A[i] < B[j])
35         {
36             i++;
37         }
38         else
39         {
40             j++;
41         }
42     }
43
44     if (matches == n)
45     {
46         cout << matches - 1 << endl;
47     }
48     else
49     {
50         cout << matches + 1 << endl;
51     }
52     return 0;
53 }
```

Output:

4	6
1 2 3 4	3 5 7 11 5 8
1 2 3 3	5 7 11 10 5 8
4	6

Verdict:

67179649	fardinnuman	A	Accepted	74	729	C++	2 months ago
----------	-------------	---	----------	----	-----	-----	--------------

Discussion:

- **Time Complexity:** $O(n \log n)$ due to sorting both arrays
- **Space Complexity:** $O(1)$, ignoring input arrays

B. Mark and Toys

Problem Statement:

Mark wants to buy as many toys as possible within a fixed budget. Each toy has a price and each toy can be bought only once. Given a list of toy prices and a total budget, the task is to determine the maximum number of toys Mark can purchase without exceeding his budget

Theory/Approach:

The optimal strategy is to always buy the cheapest toys first to maximize the count. So, the array of prices is sorted in ascending order. Then, starting from the smallest price, we keep subtracting the price from the budget as long as the remaining budget allows. Each successful purchase increases the count. The process stops when the next toy cannot be afforded. This greedy approach ensures the maximum number of toys is bought

Code:

```
G++ b.cpp > ...
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main()
6  {
7      int n, k;
8      cin >> n >> k;
9
10     vector<int> arr(n);
11     for (int i = 0; i < n; i++)
12     {
13         cin >> arr[i];
14     }
15
16     sort(arr.begin(), arr.end());
17
18     int count = 0;
19     for (int i = 0; i < n; i++)
20     {
21         if (k >= arr[i])
22         {
23             k -= arr[i];
24             count++;
25         }
26         else
27         {
28             break;
29         }
30     }
31
32     cout << count << endl;
33     return 0;
34 }
```

Output:

```
7 50
1 12 5 111 200 1000 10
4
```

Verdict:

67179416	fardinnuman	B	Accepted	480	472	C++	2 months ago
----------	-------------	---	----------	-----	-----	-----	--------------

Discussion:

- **Time Complexity:** $O(n \log n)$ due to sorting
- **Space Complexity:** $O(n)$ for storing the array

C. Maps - STL

Problem Statement:

Given a series of queries related to student marks, each student is identified by a name and can have marks added multiple times. The queries allow adding marks to a student, resetting a student's marks to zero, or printing the current marks of a student. If a student has no recorded marks, the output should be zero. The task is to process all queries and produce outputs for the print operations

Theory/Approach:

A map is used to store student names as keys and their total marks as values. For each query, if the type is 1, the given marks are added to the student's existing total. If the type is 2, the student's marks are reset to zero. If the type is 3, the current marks of the student are printed and if the student does not exist in the map, it automatically returns zero. This approach efficiently handles all operations using direct key-based access

Code:

```
G++ c.cpp > ...
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main()
6  {
7      int q;
8      cin >> q;
9
10     map<string, int> marks;
11
12     while (q--)
13     {
14         int type;
15         cin >> type;
16
17         if (type == 1)
18         {
19             string name;
20             int score;
21
22             cin >> name >> score;
23             marks[name] += score;
24         }
25         else if (type == 2)
26         {
27             string name;
28             cin >> name;
29             marks[name] = 0;
30         }
31         else if (type == 3)
32         {
33             string name;
34             cin >> name;
35             cout << marks[name] << endl;
36         }
37     }
38
39     return 0;
40 }
```

Output:

```
7
1 Jesse 20
1 Jess 12
1 Jess 18
3 Jess
30
3 Jesse
20
2 Jess
3 Jess
0
```

Verdict:

67180356	fardinnuman	C	Accepted	1117	638	C++	2 months ago
----------	-------------	---	----------	------	-----	-----	--------------

Discussion:

- **Time Complexity:** $O(\log n)$ per query due to map operations
- **Space Complexity:** $O(n)$ for storing student records

D. Balanced Brackets

Problem Statement:

Given several strings consisting of brackets `()`, `{}` and `[]`, determine whether each string is balanced. A string is balanced if every opening bracket has a corresponding closing bracket of the same type and the brackets enclosed within a matched pair are also properly balanced

Theory/Approach:

To approach is to use a stack to track unmatched opening brackets. Iterate through each character: push opening brackets onto the stack, and for closing brackets, check if the top of the stack matches the corresponding opening bracket. If it matches, pop the stack; otherwise, the string is unbalanced. After processing the string, if the stack is empty, the string is balanced; otherwise, it is not. This ensures proper nesting and type matching

Code:

```
G++ d.cpp > ...
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main()
6  {
7      int t;
8      cin >> t;
9
10     unordered_map<char, char> match = {
11         {'}', '('},
12         {'}', '{'},
13         {'}', '['},
14     };
15
16     while (t--)
17     {
18         string s;
19         cin >> s;
20
21         stack<char> stk;
22         bool ok = true;
23
24         for (char ch : s)
25         {
26             if (ch == '(' || ch == '{' || ch == '[')
27             {
28                 stk.push(ch);
29             }
30             else
31             {
32                 if (stk.empty() || stk.top() != match[ch])
33                 {
34                     ok = false;
35                     break;
36                 }
37                 stk.pop();
38             }
39         }
40
41         if (!stk.empty()) ok = false;
42
43         cout << (ok ? "YES\n" : "NO\n");
44     }
45
46     return 0;
47 }
```

Output:

```
3
{[(())]}
YES
{[(())]}
NO
{{[[((()))]]}}
YES
```

Verdict:

67189287	fardinnuman	D	Accepted	485	1030	C++	2 months ago
----------	-------------	---	----------	-----	------	-----	--------------

Discussion:

- **Time Complexity:** $O(\log n)$ per query due to map operations
- **Space Complexity:** $O(n)$ for storing student records

E. Between Two Sets

Problem Statement:

You are given two arrays of integers. The task is to find how many integers exist such that all elements of the first array are factors of that integer and the integer itself is a factor of all elements of the second array. These integers are said to lie “between” the two arrays

Theory/Approach:

The approach is to check all possible integers between the maximum value of the first array and the minimum value of the second array. For each candidate number, we verify two conditions: it must be divisible by every element of the first array and every element of the second array must be divisible by it. If both conditions are satisfied, it is counted as a valid number. This brute-force method works efficiently due to the limited range of possible values

Code:

```
G++ e.cpp > ...
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main()
6  {
7      int n, m, count = 0;
8      cin >> n >> m;
9
10     int a[n], b[m];
11     for (int i = 0; i < n; i++)
12         cin >> a[i];
13     for (int i = 0; i < m; i++)
14         cin >> b[i];
15
16     for (int x = a[n - 1]; x <= b[0]; x++)
17     {
18         int ok = 1;
19
20         for (int i = 0; i < n; i++)
21             if (x % a[i])
22                 ok = 0;
23
24         for (int i = 0; i < m; i++)
25             if (b[i] % x)
26                 ok = 0;
27
28         if (ok)
29             count++;
30     }
31
32     cout << count;
33
34     return 0;
35 }
```

Output:

```
2 3
2 4
16 32 96
3
```

Verdict:

67180188	fardinnuman	E	Accepted	85	561	C++	2 months ago
----------	-----------------------------	-------------------	----------	----	-----	---------------------	--------------

Discussion:

- **Time Complexity:** $O((\max(b) - \min(a)) \times (n + m))$
- **Space Complexity:** $O(n + m)$ for storing the arrays

“2. STL Problems”

2.1 | Unique and Sorted Elements (Vector)

Problem Statement:

Given N integers ($1 \leq N \leq 10^5$), remove all duplicates and print the numbers in ascending order

Theory/Approach:

1. Store numbers in a vector
2. Sort the vector in ascending order
3. Use the unique algorithm to remove consecutive duplicates
4. Erase the redundant elements from the vector

Code:

```
STL > C++ 1.cpp > ...
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      int n;
7      cin >> n;
8      vector<int> nums(n);
9      for (int i = 0; i < n; i++)
10         cin >> nums[i];
11
12     sort(nums.begin(), nums.end());
13     nums.erase(unique(nums.begin(), nums.end()), nums.end());
14
15     for (int num : nums)
16         cout << num << " ";
17     cout << endl;
18     return 0;
19 }
```

Output:

```
8
5 3 8 3 5 2 8 1
1 2 3 5 8
```

Discussion:

- Sorting ensures that duplicates are consecutive, making unique effective.
- Complexity: $O(n \log n)$ due to sorting.
- Efficient for large inputs.

2.2 | Character Frequency Counter (Map)

Problem Statement:

Given a string S ($1 \leq |S| \leq 10^5$), count the frequency of each character

Theory/Approach:

1. Use map $\langle \text{char}, \text{int} \rangle$ to store each character and its frequency
2. Traverse the string and increment the count for each character

Code:

```
STL > C++ 2.cpp > ...
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      string s;
7      getline(cin, s);
8      map<char, int> freq;
9
10     for (char c : s)
11         freq[c]++;
12     for (auto &p : freq)
13         cout << "'" << p.first << "': " << p.second << endl;
14
15     return 0;
16 }
```

Output:

```
Hello World
' ': 1
'H': 1
'W': 1
'd': 1
'e': 1
'l': 3
'o': 2
'r': 1
```

Discussion:

- Maps automatically sort keys in ascending order
- Complexity: $O(n \log n)$ due to map insertions
- Useful to quickly count occurrences for any type of input

2.3 | Unique Elements Finder (Set)

Problem Statement:

Given N integers, find all unique elements using a set.

Theory/Approach:

1. Insert all elements into a set <int>
2. Sets automatically discard duplicates and store elements in order

Code:

```
STL > G++ 3.cpp > ...
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      int n;
7      cin >> n;
8      set<int> unique_set;
9
10     for (int i = 0; i < n; i++)
11     {
12         int num;
13         cin >> num;
14         unique_set.insert(num);
15     }
16
17     for (int num : unique_set)
18         cout << num << " ";
19     cout << "\nTotal unique elements: " << unique_set.size() << endl;
20     return 0;
21 }
```

Output:

```
7
5 3 8 3 5 2 8
2 3 5 8
Total unique elements: 4
```

Discussion:

- Complexity: $O(n \log n)$ for N insertions
- Printing the set gives sorted unique elements directly
- Efficient alternative to sorting + unique

2.4 | Queue Operations Simulator (Queue)

Problem Statement:

Simulate a queue with commands: ENQUEUE x, DEQUEUE, FRONT, SIZE, EXIT

Theory/Approach:

1. Use STL `queue<int>` to simulate the queue
2. Handle commands with conditional statements
3. Each operation has $O(1)$ complexity

Code:

```
STL > G++ 4.cpp > ...
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      queue<int> q;
7      string command;
8
9      while (true)
10     {
11         cin >> command;
12         if (command == "ENQUEUE")
13         {
14             int x;
15             cin >> x;
16             q.push(x);
17         }
18         else if (command == "DEQUEUE")
19         {
20             if (!q.empty())
21                 q.pop();
22         }
23         else if (command == "FRONT")
24         {
25             if (!q.empty())
26                 cout << q.front();
27         }
28         else if (command == "SIZE")
29             cout << q.size();
30         else if (command == "EXIT")
31             break;
32     }
33
34     return 0;
35 }
```

Output:

```
ENQUEUE 10
ENQUEUE 20
FRONT
10
SIZE
2
DEQUEUE
FRONT
20
EXIT
```

Discussion:

- Useful for modeling real-life queuing systems
- Avoids manual array shifting for DEQUEUE, keeping operations fast

2.5 | Balanced Parentheses Checker (Stack)

Problem Statement:

Check if a string of parentheses (), {}, [] is balanced

Theory/Approach:

1. Use a stack to track opening brackets
2. For each closing bracket, check if it matches the top of the stack
3. If stack is empty at the end and all matches succeeded → balanced

Code:

```
STL > G++ 5.cpp > ...
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  bool isBalanced(string s)
5  {
6      stack<char> st;
7      for (char c : s)
8      {
9          if (c == '(' || c == '{' || c == '[')
10             st.push(c);
11         else
12         {
13             if (st.empty())
14                 return false;
15             char top = st.top();
16             st.pop();
17             if ((c == ')' && top != '(') || (c == '}' && top != '{') || (c == ']' && top != '['))
18                 return false;
19         }
20     }
21     return st.empty();
22 }
23
24 int main()
25 {
26     string s;
27     cin >> s;
28     cout << (isBalanced(s) ? "Balanced" : "Not Balanced") << endl;
29     return 0;
30 }
```

Output:

```
{[(())]}    {[()]}
```

Balanced Not Balanced

Discussion:

- Complexity: O(N), linear in string length
- Stack ensures correct nesting and pairing
- Works for multiple types of brackets simultaneously

2.6 | Sort Pairs (Pair)

Problem Statement:

Given N pairs of integers, sort them first by the first element, then by the second

Theory/Approach:

- Store pairs in vector <pair<int,int>>
- Use sort(); default comparator sorts by first, then second element

Code:

```
STL > G++ 6.cpp > ...
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      int n;
7      cin >> n;
8      vector<pair<int, int>> pairs(n);
9      for (int i = 0; i < n; i++)
10         cin >> pairs[i].first >> pairs[i].second;
11
12     sort(pairs.begin(), pairs.end());
13     for (auto &p : pairs)
14         cout << "(" << p.first << ", " << p.second << ")" << endl;
15     return 0;
16 }
```

Output:

```
5
3 4
1 2
5 1
3 1
2 2
(1, 2)
(2, 2)
(3, 1)
(3, 4)
(5, 1)
```

Discussion:

- Complexity: $O(n \log n)$ due to sorting
- Very useful for coordinate sorting, event scheduling, and mapping

“3. PRIME, GCD, LCM”

1. Almost Prime

Problem Statement:

A number is called *almost prime* if it has exactly two distinct prime divisors. For example, 6 (2×3), 18 (2×3^2) and 24 ($2^3 \times 3$) are almost prime, whereas 4 (2^2), 8 (2^3), 9 (3^2) and 42 ($2 \times 3 \times 7$) are not. Given an integer n , count how many almost prime numbers exist between 1 and n , inclusive

Theory/Approach:

- Iterate through all numbers from 2 to n
- For each number, count its distinct prime divisors:
 - Check divisibility starting from 2 up to the square root of the number
 - Keep track of distinct primes using a counter
- If a number has exactly two distinct prime divisors, increment the almost prime counter
- Return the total count

Code:

```
3 > G+ almostPrime.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  bool isAlmostPrime(int num)
5  {
6      int count = 0;
7      for (int i = 2; i * i <= num; i++)
8      {
9          if (num % i == 0)
10         {
11             count++;
12             while (num % i == 0)
13                 num /= i;
14         }
15     }
16     if (num > 1)
17         count++;
18     return count == 2;
19 }
20
21 int main()
22 {
23     int n;
24     cin >> n;
25     int result = 0;
26     for (int i = 2; i <= n; i++)
27     {
28         if (isAlmostPrime(i))
29             result++;
30     }
31     cout << result << endl;
32     return 0;
33 }
```

Output:

```
10 21
2 8
```

Verdict:

369457340	Apr/03/2026 17:41 UTC+6	fardinnuman	26A - Almost Prime	C++17 (GCC 7-32)	Accepted	62 ms	100 KB
-----------	-------------------------	-------------	--------------------	------------------	----------	-------	--------

Discussion:

- Time Complexity: $O(n\sqrt{n})$, since we factorize each number up to its square root
- Space Complexity: $O(1)$, apart from loop variables

2. GCD

Problem Statement:

Given a number n , consider all integers from 1 to n . Among all pairs of distinct integers (a, b) , find the maximum possible greatest common divisor (GCD). For multiple test cases, return the maximum GCD for each n

Theory/Approach:

The maximum GCD of two numbers in the range $1 \dots n$ is always $n/2$ (integer division). This is because the pair $(n/2, n)$ gives the highest common divisor: n is divisible by $n/2$. Therefore, for each test case, we can directly return $n/2$ without checking all pairs, which is a simple greedy observation based on number theory

Code:

```
G++ gcd_maximumGCD.cpp > ...
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      int t;
7      cin >> t;
8      while (t--)
9      {
10         int n;
11         cin >> n;
12         cout << n / 2 << "\n";
13     }
14     return 0;
15 }
```

Output:

```
2 3 5
1
2
```

Verdict:

359200884	Jan/22/2026 18:30 ^{UTC+6}	fardinnuman	1370A - Maximum GCD	C++17 (GCC 7-32)	Accepted	31 ms	0 KB
-----------	------------------------------------	-------------	---------------------	------------------	----------	-------	------

Discussion:

- Time Complexity: $O(t)$, since we calculate $n/2$ for each of the t test cases
- Space Complexity: $O(1)$, as no extra space is needed beyond input variables

3. LCM

Problem Statement:

Given an integer n , find two positive integers a and b such that $a + b = n$ and the least common multiple (LCM) of a and b is minimized. If multiple pairs give the same minimum LCM, any pair can be output

Theory/Approach:

- The LCM of two numbers a and b is minimized when a and b share the maximum possible common factor
- To achieve this, we look for the largest divisor of n (less than n) as a , and set $b = n - a$.
- If n is even, the best pair is $(n/2, n/2)$, giving $\text{LCM} = n/2$
- If n is odd, we iterate through divisors up to $\sqrt{n}$ to find the divisor d that minimizes $\text{LCM}(d, n-d)$ efficiently

Code:

```
G+ lcm_minimumLCM.cpp > ...
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      int t;
7      cin >> t;
8      while (t--)
9      {
10         long long n;
11         cin >> n;
12         long long a = 1;
13         for (long long i = 1; i * i <= n; i++)
14         {
15             if (n % i == 0)
16             {
17                 long long d1 = i, d2 = n / i;
18                 if (d1 <= n / 2)
19                     a = max(a, d1);
20                 if (d2 <= n / 2)
21                     a = max(a, d2);
22             }
23         }
24         cout << a << " " << n - a << "\n";
25     }
26 }
```

Output:

```
4 2 9 5 10
1 1
3 6
1 4
5 5
```

Verdict:

359201343	Jan/22/2026 18:34 ^{UTC+6}	fardinnuman	1765M - Minimum LCM	C++17 (GCC 7-32)	Accepted	62 ms	0 KB
-----------	------------------------------------	-------------	---------------------	------------------	----------	-------	------

Discussion:

- Time Complexity: $O(\sqrt{n})$ per test case
- Space Complexity: $O(1)$, as only a few variables are needed for computation